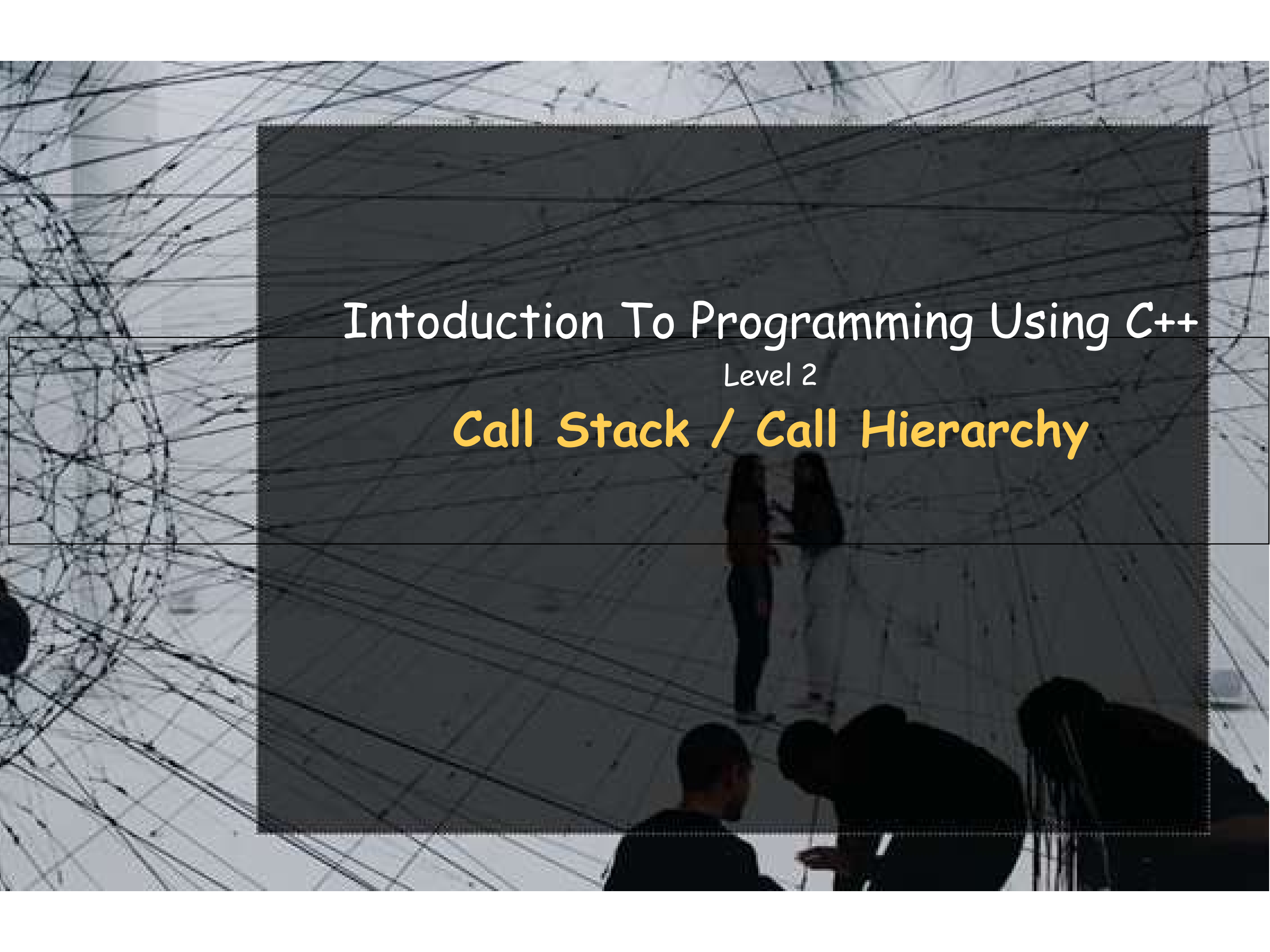


The background image shows a blurred computer monitor. On the left, a sidebar of a code editor is visible with various icons and a list of files. The main area of the screen displays C++ code with syntax highlighting. A call stack or call hierarchy window is open, showing a tree-like structure of function calls. The text 'Introduction To Programming Using C++' is overlaid in a large, yellow, sans-serif font, and 'Level 2' is overlaid in a smaller, white, sans-serif font below it.

Introduction To Programming Using C++ Level 2

Call Stack / Call Hierarchy



Introduction To Programming Using C++

Level 2

Call Stack / Call Hierarchy

Definition

- ◆ Call Stack is the stack that keeps track of the active Function Calls during program execution.
- ◆ Call Hierarchy shows the relationship between Functions and which Function calls another Function.

Explanation

- If we have several Functions, and one Function calls another Function, which then calls another Function, this creates a Call Stack during execution.
- Each Function that calls another Function waits for the called Function to finish before continuing its own execution.
- Each active Function has a Stack Frame that contains information related to its current execution.
- The Call Stack follows the Last In, First Out (LIFO) principle.

Procedures

1. `main()` starts executing.
 2. `main()` calls `Function1()`.
 3. `Function1()` calls `Function2()`.
 4. `Function2()` calls `Function3()`.
 5. `Function3()` calls `Function4()`.
 6. `Function4()` finishes execution and is removed from the Call Stack.
7. The program returns to `Function3()`, then `Function2()`, then `Function1()`, and finally `main()`.

💡 Example

```
void Function4()
{ cout << "Hi I'm function4 " << endl; }
void Function3()
{ Function4(); }
void Function2()
{ Function3(); }
void Function1()
{ Function2(); }
int main()
{
    Function1();
    return 0;
}
```

➡ Result

Hi I'm function4

Note

■ There is normally one active Stack Frame at the top of the Call Stack at any given moment.

■ The order of entering the Call Stack is:

```
main()
↓
Function1()
↓
Function2()
↓
Function3()
↓
Function4()
```

■ The order of leaving the Call Stack is the reverse:

```
Function4()
↓
Function3()
↓
Function2()
↓
Function1()
↓
main()
```

⊘ Warning

- ◆ Call Stack and Call Hierarchy are related but not exactly the same.
 - ▨ Call Stack shows the Functions that are currently active during execution.
 - ▨ Call Hierarchy shows the relationships between Function Calls in the code.



Comparison

▨ Call Stack

- ✓ Tracks active Function Calls during program execution.
- ✓ Follows Last In, First Out (LIFO).
- ✓ Changes as Functions are called and return.
- ✓ Useful during Debugging.

▤ Call Hierarchy

- ✓ Shows the calling relationships between Functions.
- ✓ Helps identify which Function calls another Function.
- ✓ Can be viewed in Visual Studio using `View Call Hierarchy`.
- ✓ Helps understand the structure and flow of a program.

✦ Summary

- ✓ Call Stack keeps track of active Function Calls.
- ✓ Each Function Call creates a Stack Frame.
- ✓ The last Function to enter the Call Stack is the first to leave.
- ✓ Call Stack follows Last In, First Out (LIFO).
- ✓ Call Hierarchy shows the relationships between Functions.

Thank you 😊